A6 Coursework

Alex Blake Martín, apb86

Easter Term 2026, word count: 2710

1 Ellipticity estimation

1.1 Weak lensing theory

The following discussion closely follows that from Bartelmann and Schneider 2001. Gravitational lensing, the deflection of light by matter, distorts observed images of galaxies. In the weak lensing regime, the transformation between unlensed coordinates (x_u, y_u) and lensed coordinates (x_l, y_l) can be described by a linear transformation

$$\begin{pmatrix} x_u \\ y_u \end{pmatrix} = \begin{pmatrix} 1-\gamma_1-\kappa & -\gamma_2 \\ -\gamma_2 & 1+\gamma_1-\kappa \end{pmatrix} \begin{pmatrix} x_l \\ y_l \end{pmatrix}, \tag{1}$$

where the two components of shear that relate to the stretching of the image can be combined into a single complex number $\gamma = \gamma_1 + i\gamma_2$ and the convergence κ describes changes in size and brightness. Lensing shear induces a change in a galaxy's measured ellipticity. Ellipticity can be measured by using a profile with a well-defined ellipticity $\epsilon = \epsilon_1 + i\epsilon_2$ with modulus

$$|\epsilon| = \frac{a-b}{a+b} = \frac{1-q}{1+q}, \tag{2}$$

where the semi-major and semi-minor axis lengths a and b respectively, and their ratio is given by $q = b/a$.

Another definition is based on the second moments of the brightness of an image of a galaxy:

$$Q_{ij} = \frac{\int d^2x I(\mathbf{x}) W(\mathbf{x}) x_i x_j}{\int d^2x I(\mathbf{x}) W(\mathbf{x})}, \tag{3}$$

where $I(\mathbf{x})$ denotes the galaxy image brightness, $W(\mathbf{x})$ denotes an optional weight function, and the coordinate origin is at the centroid of the image. Using this definition we can form a measure of the ellipticity from quadrupole moments:

$$\epsilon = \epsilon_1 + i\epsilon_2 = \frac{Q_{11} - Q_{22} + 2iQ_{12}}{Q_{11} + Q_{22} + 2(Q_{11}Q_{22} - Q_{12}^2)^{1/2}}. \tag{4}$$

Taking an ensemble average of the ellipticity in the weak lensing limit and under the assumption that source galaxies are not intrinsically aligned shows that $\langle \epsilon \rangle \approx \gamma/(1-|\gamma|^2) \approx \gamma$ when $|\gamma| \ll 1$, justifying our use of the ensemble-averaged ellipticity as a shear estimator (Seitz and Schneider 1997).

1.2 Second moments approach

As a first approach to estimating galaxy ellipticities, we measure ϵ_1 and ϵ_2 using the second moments of the galaxy image brightnesses with a weight function that is unity everywhere in the unweighted case, and a circular Gaussian with full width at half-maximum (FWHM) of four pixels in the weighted case, which down weights contributions further from the image centre that are mainly sourced by noise, but which also contain flux from the outer portions of the galaxy. The FWHM relates to the standard deviation of a Gaussian distribution via the relation

$$\mathrm{FWHM} = 2\sqrt{2\ln 2}\,\sigma. \tag{5}$$

This method can produce unphysical values of the ellipticity i.e. whenever $|\epsilon| > 1$, which we disregard in the comparison. This arises when the denominator in Equation [4] is small or negative due to noise. Furthermore, the weighted second moments approach down weights the contribution to ellipticity from pixels further from the galaxy centre, which is intended to suppress noise. This also suppresses the contribution from flux from the outer part of the galaxy. A circularly-symmetric weight function will reduce the asymmetry of an observed galaxy compared to its intrinsic shape, lowering the measured ellipticity. As a result, the estimate $\gamma \approx \langle\epsilon\rangle$ is biased to lower values and requires a multiplicative bias-correction factor to be applied; however, following the brief's instruction to neglect shear estimation biases for these high signal-to-noise simulations, this correction is not applied. There is a second multiplicative bias contribution that arises from the estimator's nonlinear combination of pixel noise given by $m \approx -N_{\mathrm{pix}}\sigma^2_{\mathrm{pix}}/F^2$, where $N_{\mathrm{pix}}$is the number of pixels in a galaxy stamp, σ^2_{pix} is the pixel variance, and F is the total galaxy flux (Kaiser 2000). This quantity is small for these high signal-to-noise simulations and so is not corrected for. In real data, there would be a point-spread function (PSF) which smears the galaxy image, introducing both a multiplicative and an additive bias from the PSF anisotropy (Kaiser et al. 1995, KSB).

The centroid of the unweighted image can be simply calculated using the expression:

$$\bar{x}_i = \frac{\int d^2x I(\mathbf{x})x_i}{\int d^2x I(\mathbf{x})}. \tag{6}$$

The centroid of the weighted image is more complex, and is determined by an iterative scheme that initialises the weighted image centroid at the unweighted centroid, centres the Gaussian weight function there, recomputes the centroid, and repeats until convergence to within 10^{-3} pixels, a threshold chosen to balance centroid accuracy with speed of convergence.

1.3 The Hirata-Seljak-Mandelbaum (HSM) algorithm

The HSM algorithm was first introduced in Hirata and Seljak 2003 (HS03), expanding on the adaptive moments method from Bernstein and Jarvis 2002 (BJ02) to introduce a novel PSF correction approach termed re-Gaussianisation, and was first tested on real data in Mandelbaum et al. 2005 (M05). It is implemented in the package `GalSim`[1] (Rowe et al. 2015) through the `HSM` module[2]. The algorithm adaptively fits an elliptical weight function individually to each galaxy image. We use the `observed_shape.g1` and `observed_shape.g2` methods which use the reduced shear convention $g = \frac{\gamma}{1-\kappa}$. This differs from the ϵ prescription, but they are both unbiased ensemble estimates of the reduced shear for randomly oriented sources.

[1] https://galsim-developers.github.io/GalSim/_build/html/index.html
[2] https://galsim-developers.github.io/GalSim/_build/html/hsm.html

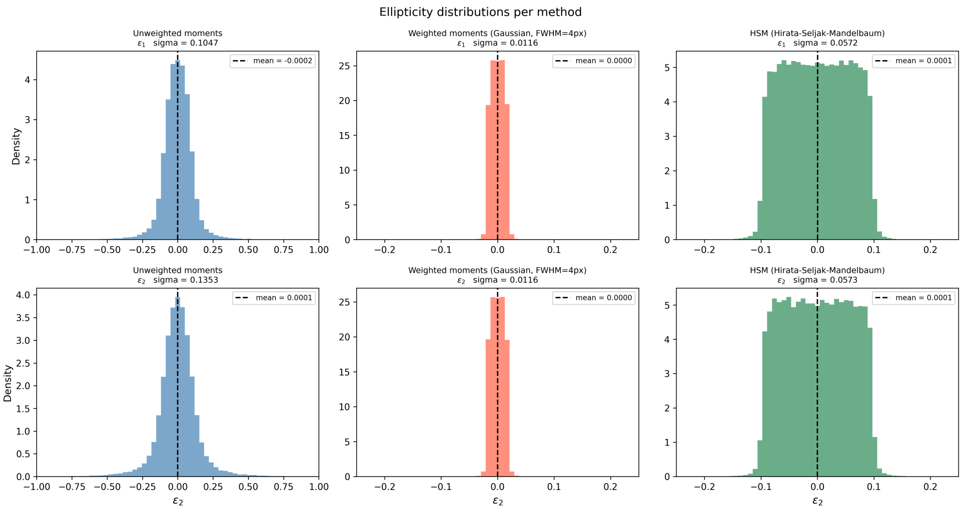

Figure 1: Histograms of ellipticity components for three different methods.

1.4 Comparison of methods

It is first important to note that the unweighted second moments method results in 7649 galaxies with unphysical or invalid ellipticities, which are removed from the comparison, while the weighted second moments method and the HSM algorithm have the much more desirable property that almost all ellipticities measured are valid. The weighted moments approach rejects just three galaxies, with the HSM algorithm rejecting none. This suggests that the unweighted moments method produces the most noisy estimates of ellipticity of the three methods. Histograms of the distributions of the components of ellipticity for each method are shown in Figure [1]. In general, each method produces similar distributions for the two components of shear, with means that almost exactly coincide at zero and standard deviations that are almost identical, with the exception of the unweighted moments method, which has a slightly broader distribution in the second component ($\sigma = 0.1353$) than the first ($\sigma = 0.1047$). The fact that the means of each distribution are all very close to zero demonstrates that none of the methods produce ellipticity estimates containing additive bias. It can also be seen that the distribution is very tight for the weighted moments method, with a standard deviation of $\sigma = 0.0116$ in each component of ellipticity. This implies that the FWHM used in the Gaussian weight function is smaller than the typical size of a galaxy. The result is that only the core of each galaxy is sampled, which is less elongated than the full galaxy profile.

Figure [2] shows a scatter plot of the two components of ellipticity for each method. It can be seen that the HSM algorithm produces the least noisy estimates of ellipticity of the three methods, with nearly all points confined to a central square region spanning the interval $[-0.25, 0.25]$ in each component. By contrast, despite the very tight distribution exhibited by the weighted moments method, several points deviate significantly from the central cluster, which may be an indication that it is a more noisy ellipticity estimator than the HSM algorithm. Further supporting the claim that the unweighted moments method produces the noisiest estimate of ellipticity is the fact that many of the points in this scatter plot deviate significantly from the densely-populated central region, indicating a tendency to predict extreme ellipticities. This can be understood by returning to its definition, where the contributions from noise in the background outside a galaxy or other objects very close to the galaxy on the sky are given equal contributions to the centre of the galaxy when computing the ellipticity and can inflate one or both components.

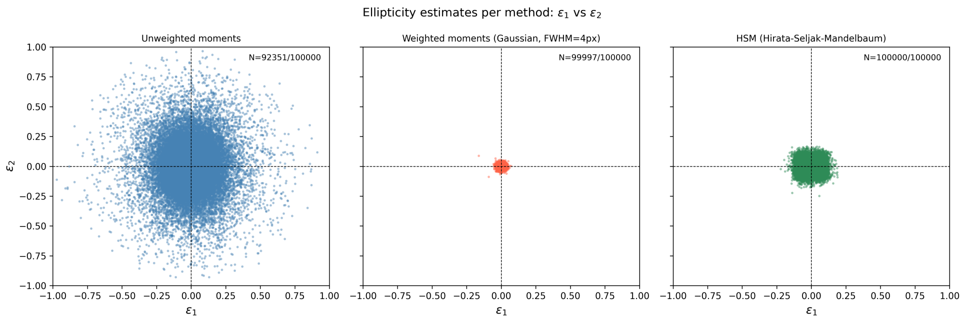

Figure 2: Scatter plots of the two components of ellipticity for the three different methods.

In Figure [3] we show a direct comparison between the two moments methods with the HSM algorithm, we report the slope of the ordinary least-squares regression line and the Pearson correlation coefficient for each comparison. We see that there is a mild correlation between the predictions made by the HSM algorithm and the unweighted moments method with correlation coefficients of 0.557 and 0.426 in ϵ_1 and ϵ_2 respectively. The correlation is much stronger in the case of the weighted moments method, which registers correlation coefficients of 0.977 and 0.978 in ϵ_1 and ϵ_2 respectively. Conversely, the slopes of the regression lines for the unweighted case are 1.024 ± 0.005 and 1.009 ± 0.007 for ϵ_1 and ϵ_2 respectively, while in the weighted case, the slopes are 0.197 ± 0.000. These confirm the previous discussion that the unweighted moments method doesn't produce an additional multiplicative bias compared to the HSM algorithm, even though its estimates are very noisy and have a large amount of scatter, while the weighted moments method has an extra multiplicative bias despite being a less noisy estimate. The weighted moments method specifically requires a correction factor of ≈ 5 as a result of the FWHM of the weight function being significantly smaller than the half-light radius of a typical galaxy.

We conclude that the HSM algorithm is the most suitable estimator for this method in the high signal-to-noise case and without correcting for biases, as the brief states. This is a result of its lower noise compared to the unweighted moments method, and smaller multiplicative bias than for the weighted moments estimator.

2 Navarro-Frenk-White (NFW) dark matter haloes

2.1 NFW theory

The Navarro-Frenk-White (NFW) profile is an analytic formula for the radial density profile of dark matter haloes that depends on just two halo parameters:

$$\frac{\rho(r)}{\rho_{\mathrm{crit}}} = \frac{\delta_c}{r/r_s(1+r/r_s)^2}, \tag{7}$$

where r_s is a characteristic radius, δ_c is a dimensionless density parameter, and $\rho_{\mathrm{crit}} = 3H^2/8\pi G$ is the critical density. This discovery was motivated by the observation from N-body simulations that dark matter haloes formed through hierarchical structure formation are self-similar with the relationship between parameters strongly linked to the underlying cosmology (Navarro et al. 1995; Navarro et al. 1997).

2.2 Tangential shear measurements around NFW haloes

The component of shear that is tangential to the separation vector between the source and the lens is given by:

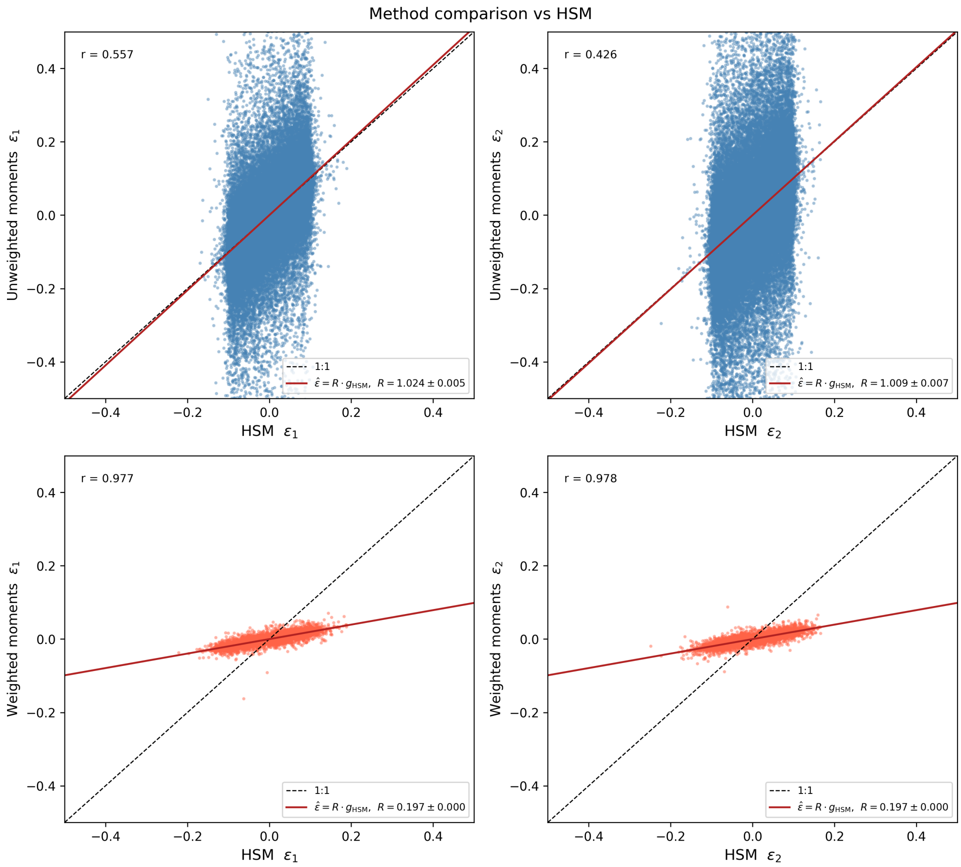

Figure 3: Scatter plots of ellipticity estimates for moments methods against those for the HSM algorithm.

$$\gamma_t = -(g_1 \cos 2\phi + g_2 \sin 2\phi). \tag{8}$$

The other component of shear, known as the cross shear, vanishes for purely gravitational shears. This provides a useful null test, where the measured cross shear should be consistent with zero for all separations. A nonzero cross shear would indicate the presence of systematics that are not accounted for affecting the result. The cross shear is given by the expression:

$$\gamma_\times = (g_1 \sin 2\phi - g_2 \cos 2\phi). \tag{9}$$

In the weak lensing limit, sufficiently far away from the halo centre where the convergence $\kappa \approx 0$, the shear, reduced shear, and ellipticity coincide i.e. $\gamma \approx g \approx \epsilon$, so we can replace the reduced shear g with ellipticity ϵ in Equations [8] and [9]. Combining the angular positions of the source galaxies and haloes with the measured galaxy ellipticities, we can thus compute the tangential and cross shears for each source-lens pair in the simulation. We bin the tangential shear estimates in 10 log-spaced bins of angular separation, computed using the Euclidean distance measure from 100 to 3600 arcseconds and plot the result in Figure [4] for the per-bin unweighted sample mean. A more optimal choice of weighting would be to use inverse shape noise variance weighting which minimises the variance in the final estimate. In this case, this would also correspond to uniform weighting, since the shape noise is the same for each galaxy as they are all drawn from the same galaxy population. Other weighting schemes exist, such as using a redshift dependent weight based on the critical surface mass density (M05). For a more realistic simulation or dataset, shape noise depends on the signal-to-noise variance, galaxy size, and the PSF. We observe that the unweighted moments method and the HSM algorithm exhibit a very strong agreement, with tighter error bars for the HSM algorithm, further supporting the claim that the unweighted moments method yields noisy estimates of ellipticity. There is a significant discrepancy between these two methods and the predictions made by the weighted moments method, which consistently predicts much smaller values of shear. The factor by which the shear is suppressed is ≈ 0.2, which is very close to the slope of the regression line between this method and the HSM algorithm, indicating a good consistency of this estimator. While the error bars are comparable to those of the HSM algorithm, indicating a low-noise estimate, the narrow Gaussian weight function is strongly biasing the shear estimate towards lower values. We thus use estimates made by the HSM algorithm in the remainder of this work due to its low bias and low noise predictions.

We also show the same plot for the cross shear with angular separation using the same binning scheme in Figure [5]. The weighted moments method and HSM algorithm give cross shears that are consistent with zero for all angular separations, indicating that there are likely to be no systematics present in the analysis that contribute on a similar level as the variance of each method. The same is not true for the unweighted moments method, for which some bins are discrepant from zero at the $1 - 2\sigma$ level. Given the large error bars, this discrepancy is likely to arise from noise in the estimator. The error bars shrink with angular separation as a result of the increasing number of lens-source pairs with distance from the lens, demonstrating increased signal to noise in the outer bins.

Assuming all source galaxies are at the same redshift, we compute the posterior probability density on the lens mass used for the identical NFW haloes and the source redshift by generating theoretical predictions for the tangential shear profile at our angular separation bin centres of an NFW halo at redshift $z = 0.3$ with a concentration of 4 and underlying cosmological parameters $\Omega_m = 0.3$, and $\Omega_\Lambda = 0.7$. Using the HSM estimates for tangential shear profile, we compute a simple Gaussian likelihood:

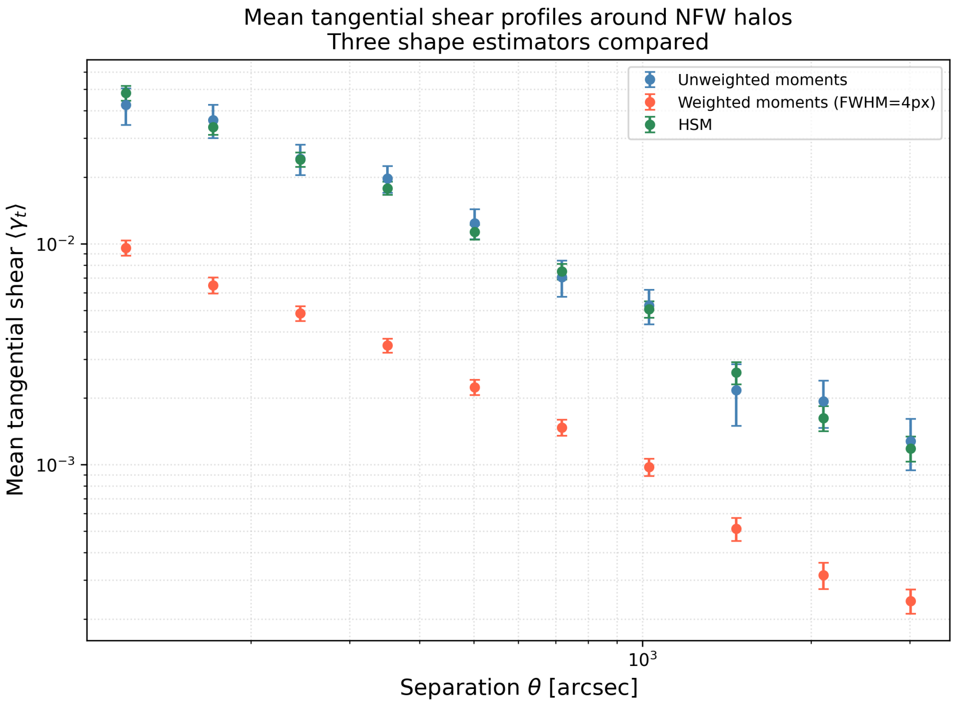

Figure 4: Mean tangential shear profiles with separation.

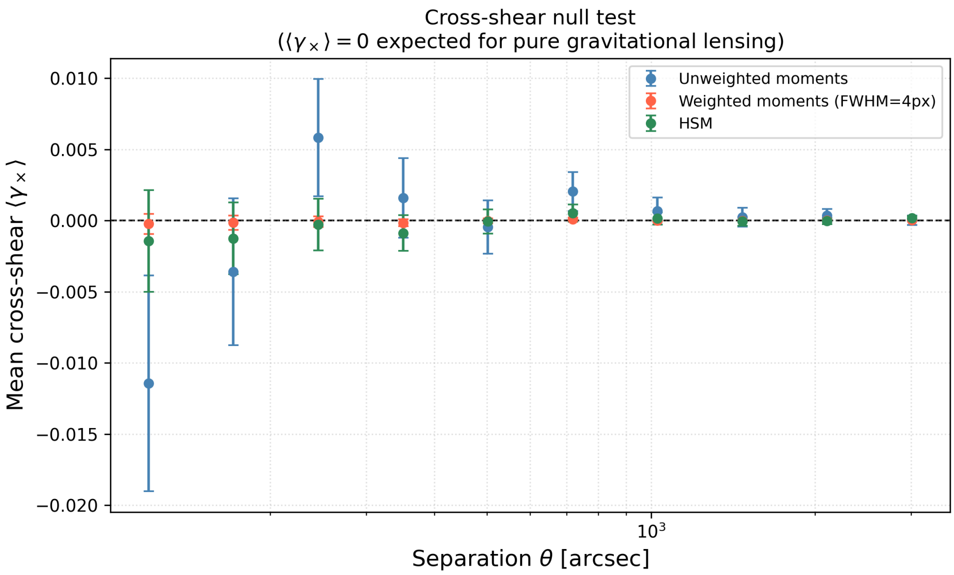

Figure 5: Mean cross shear profiles with separation.

$$\ln L = -\frac{1}{2}\sum_{k=1}^{10}\frac{(\gamma_{t,k} - \gamma_{\mathrm{theory},k})^2}{\sigma^2_{\gamma_{t,k}}}, \tag{10}$$

where the label k indexes the angular separation bin and $\sigma^2_{\gamma_{t,k}}$ is the variance of the shear estimate in each bin. We then compute the posterior with flat priors on a grid $13.8 \leq \log_{10} M \leq 15.2$, $0.3 < z_s < 0.8$ at 10^6 uniformly-spaced points. These ranges were chosen to reflect the fact that halo masses span a broad dynamic range, while the halo is known to be at $z = 0.3$ and that well-resolved source galaxies cannot abundantly exist at higher redshifts due to the apparent magnitude selection threshold of realistic galaxy surveys. The joint posterior on these two parameters is shown in Figure [6] with 68% and 95% credible regions. The 2D contours use the $\chi^2(\mathrm{d.f.} = 2)$ quantiles, corresponding to $\Delta\chi^2 = 2.30$ and $\Delta\chi^2 = 6.18$ respectively. Also included are the 1D marginal posteriors with central 68% credible regions. We report a *maximum a posteriori* (MAP) estimate on these parameters of $M_{\mathrm{MAP}} = 7.40 \times 10^{14} M_\odot$ and $z_{s,\mathrm{MAP}} = 0.47$. The 1D marginal distributions yield values of $M = 6.38^{+3.11}_{-1.26} \times 10^{14} M_\odot$ and $z_s = 0.452^{+0.110}_{-0.023}$. The shape of the posterior density indicates a negative correlation between the halo redshift and its mass, implying that a lower value of the halo mass requires the source galaxies to be at a greater redshift to produce a shear of the same strength. Fixing the source redshift to $z_s = 0.6$, the 1D conditional posterior probability on the halo mass is shown in Figure [7]. In this case, the MAP estimate of the halo mass is $M_{\mathrm{MAP},z_s=0.6} = 4.65^{+0.20}_{-0.16} \times 10^{14} M_\odot$. We note that this value of source redshift differs from the MAP estimate derived from the 1D marginal posterior at the level of 1.4σ.

It is important to note that the MAP estimate lies far from the boundaries of the grid on which the posterior is evaluated, indicating that the prior ranges chosen were sensible. The simple Gaussian likelihood also neglects the possibility of correlations between bins when the bin weights, assumed to be sourced only by shape noise variance within each bin, are chosen. For more realistic experiments, the likelihood should be replaced by a full multivariate Gaussian distribution with modelled non-diagonal covariance. Contributions to off-diagonal covariance include halo-to-halo mass scatter (absent here since all haloes are identical) and the mixed term arising from a single source galaxy contributing noise to multiple bins via different lens–source pairings (Schneider et al. 2002). The lensing signal was generated by a set of identical NFW haloes at the same redshift, so the NFW model used in the posterior calculation is a valid choice. A more complex simulation with variable profiles, halo masses, and halo redshifts would necessitate a model that accounts for differing halo profiles e.g. by fitting a generalised NFW profile (Zhao 1996; Power et al. 2003) and other relevant properties to each halo separately.

3 Reliability of data analysis

Reliable measurements of weak lensing depend on understanding and controlling systematic errors. The instrument PSF smears out images, causing further distortions to galaxy images. Typically, nearby point sources can be used to calibrate this effect and correct for it. Noise and PSF residuals can bias estimates of ellipticity. Estimators can be tested on realistic simulations to understand their limitations, which crucially relies on those simulations being realistic. Applying artificial shears to characterise an ellipticity estimator's bias and correct for it, called metacalibration (Huff and Mandelbaum 2017), provides a data-driven alternative. Forward modelling of noise and PSF residuals is another complementary approach. Unresolved blending of sources arising from increased imaging depth introduces errors on the order of a few percent (Nourbakhsh et al. 2022). Lensing depends on the redshift distribution of source galaxies; large numbers of galaxies measured using photometry yield poorer redshift estimates,

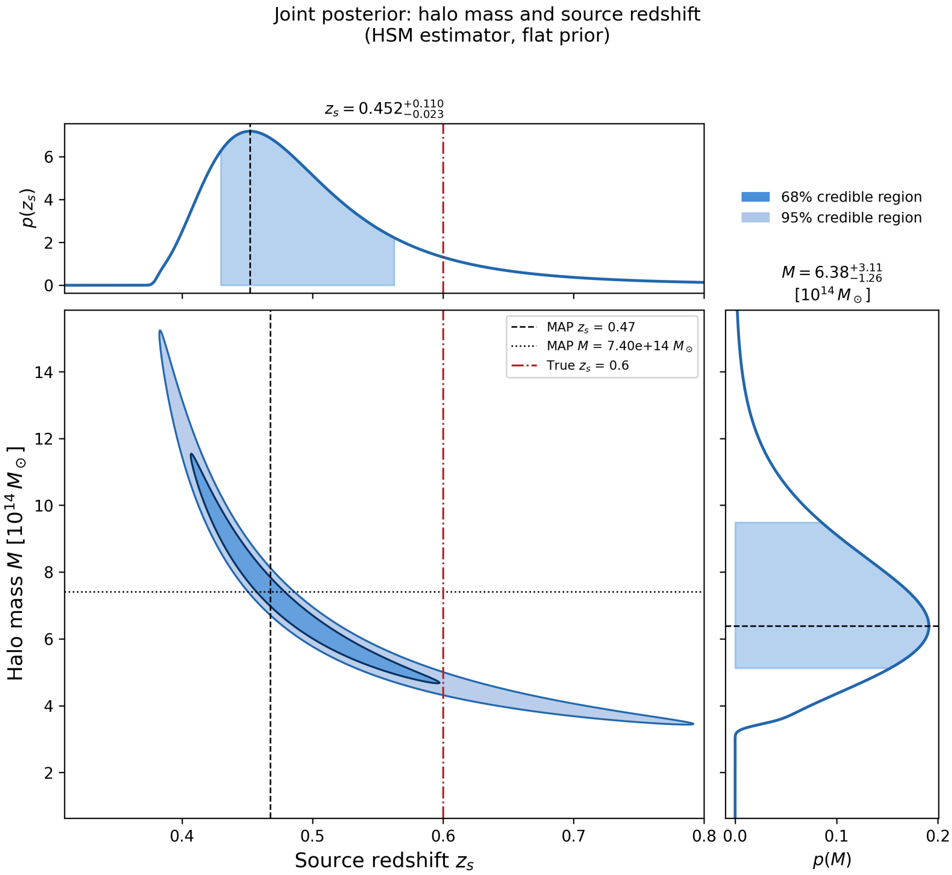

Figure 6: Posterior probability density on source galaxy redshift and halo mass for the HSM algorithm.

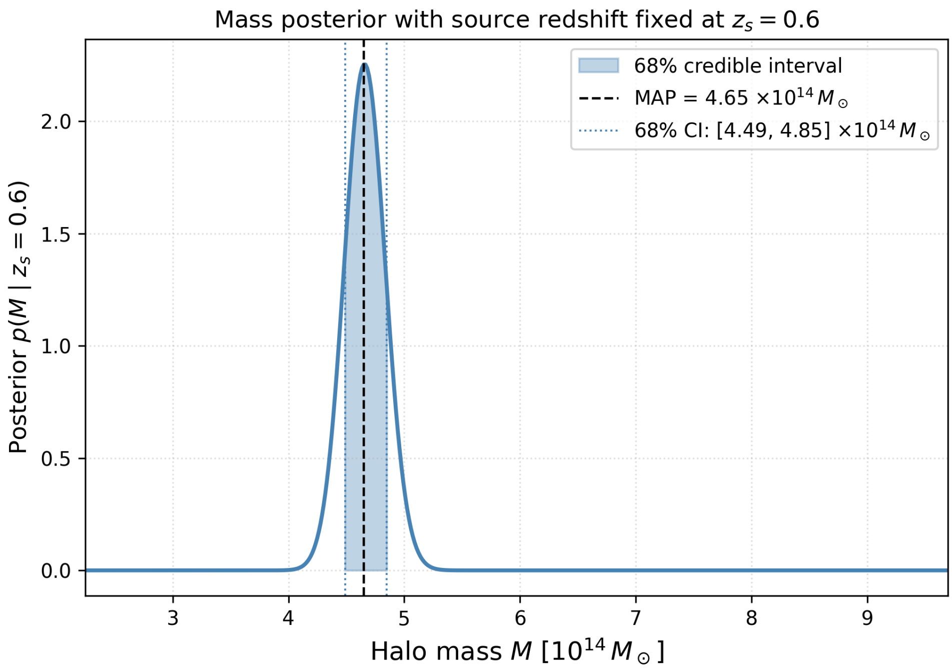

Figure 7: One dimensional conditional distribution on halo mass for fixed source galaxy redshift $z_s = 0.6$.

which can be improved by calibration against spectroscopic measurements. We initially assumed that galaxies are randomly oriented so that their average intrinsic alignment vanishes. Intrinsic alignments can however be sourced by common local tidal fields; intrinsic-intrinsic alignments can be mitigated by only correlating measurements between different source redshift bins, while gravitational-intrinsic alignments are much harder to model and account for. Poorly understood non-gravitational effects on matter distribution in galaxy clusters, such as feedback from active galactic nuclei (AGNs), supernovae, and star formation, will affect the reliability of simulations used during data analysis. In CMB lensing, residual signals from unremoved foregrounds can bias the final result. Finally, to avoid observer biases, weak lensing measurements ought to pass a series of comprehensive null tests designed *a priori* to ensure that observations are resilient to modelling choices.

References

Bartelmann, M. and P. Schneider (Jan. 2001). "Weak gravitational lensing". In: 340.4-5, pp. 291–472. DOI: 10.1016/S0370-1573(00)00082-X. arXiv: astro-ph/9912508 [astro-ph].

Bernstein, G. M. and M. Jarvis (Feb. 2002). "Shapes and Shears, Stars and Smears: Optimal Measurements for Weak Lensing". In: 123.2, pp. 583–618. DOI: 10.1086/338085. arXiv: astro-ph/0107431 [astro-ph].

Hirata, Christopher and Uroš Seljak (Aug. 2003). "Shear calibration biases in weak-lensing surveys". In: 343.2, pp. 459–480. DOI: 10.1046/j.1365-8711.2003.06683.x. arXiv: astro-ph/0301054 [astro-ph].

Huff, Eric and Rachel Mandelbaum (Feb. 2017). "Metacalibration: Direct Self-Calibration of Biases in Shear Measurement". In: *arXiv e-prints*, arXiv:1702.02600, arXiv:1702.02600. DOI: 10.48550/arXiv.1702.02600. arXiv: 1702.02600 [astro-ph.CO].

Kaiser, Nick (July 2000). "A New Shear Estimator for Weak-Lensing Observations". In: *The Astrophysical Journal* 537.2, p. 555. DOI: 10.1086/309041. URL: https://doi.org/10.1086/309041.

Kaiser, Nick, Gordon Squires, and Tom Broadhurst (Aug. 1995). "A Method for Weak Lensing Observations". In: 449, p. 460. DOI: 10.1086/176071. arXiv: astro-ph/9411005 [astro-ph].

Mandelbaum, Rachel et al. (Aug. 2005). "Systematic errors in weak lensing: application to SDSS galaxy-galaxy weak lensing". In: 361.4, pp. 1287–1322. DOI: 10.1111/j.1365-2966.2005.09282.x. arXiv: astro-ph/0501201 [astro-ph].

Navarro, Julio F., Carlos S. Frenk, and Simon D. M. White (Aug. 1995). "Simulations of X-ray clusters". In: *Monthly Notices of the Royal Astronomical Society* 275.3, pp. 720–740. ISSN: 0035-8711. DOI: 10.1093/mnras/275.3.720. eprint: https://academic.oup.com/mnras/article-pdf/275/3/720/2805863/mnras275-0720.pdf. URL: https://doi.org/10.1093/mnras/275.3.720.

— (Dec. 1997). "A Universal Density Profile from Hierarchical Clustering". In: 490.2, pp. 493–508. DOI: 10.1086/304888. arXiv: astro-ph/9611107 [astro-ph].

Nourbakhsh, Erfan et al. (Aug. 2022). "Galaxy blending effects in deep imaging cosmic shear probes of cosmology". In: 514.4, pp. 5905–5926. DOI: 10.1093/mnras/stac1303. arXiv: 2112.07659 [astro-ph.CO].

Power, C. et al. (Jan. 2003). "The inner structure of ΛCDM haloes - I. A numerical convergence study". In: 338.1, pp. 14–34. DOI: 10.1046/j.1365-8711.2003.05925.x. arXiv: astro-ph/0201544 [astro-ph].

Rowe, B. T. P. et al. (Apr. 2015). “GALSIM: The modular galaxy image simulation toolkit”. In: *Astronomy and Computing* 10, pp. 121–150. DOI: 10.1016/j.ascom.2015.02.002. arXiv: 1407.7676 [astro-ph.IM].
Schneider, P. et al. (Dec. 2002). “Analysis of two-point statistics of cosmic shear. I. Estimators and covariances”. In: 396, pp. 1–19. DOI: 10.1051/0004-6361:20021341. arXiv: astro-ph/0206182 [astro-ph].
Seitz, C. and P. Schneider (Feb. 1997). “Steps towards nonlinear cluster inversion through gravitational distortions. III. Including a redshift distribution of the sources.” In: 318, pp. 687–699. DOI: 10.48550/arXiv.astro-ph/9601079. arXiv: astro-ph/9601079 [astro-ph].
Zhao, HongSheng (Jan. 1996). “Analytical models for galactic nuclei”. In: *Monthly Notices of the Royal Astronomical Society* 278.2, pp. 488–496. ISSN: 0035-8711. DOI: 10.1093/mnras/278.2.488. eprint: https://academic.oup.com/mnras/article-pdf/278/2/488/2793437/278-2-488.pdf. URL: https://doi.org/10.1093/mnras/278.2.488.

A Declaration on the use of generative AI

Claude code with model Opus 4.6 High was used to aid in creating plots and optimising and organising code into a package with complete documentation and testing.

B Software Package: `a6cw`

All analysis code is organised into the `a6cw` Python package (version 1.0.0), hosted at https://github.com/AlexBM173/a6-cw with documentation at https://a6-cw.readthedocs.io. The package separates reusable algorithmic code from the two Jupyter notebooks, which contain only top-level orchestration. It is installed in development mode via `pyproject.toml` (`setuptools` build backend) and comprises two submodules:

Module	**Purpose**
`a6cw.ellipticity`	Galaxy shape estimators and response calibration
`a6cw.shear`	Shear profiles, NFW theory, posterior inference

B.1 `a6cw.ellipticity`

This module implements three galaxy shape estimators and the simulation-based calibration scheme for the Gaussian-weighted moment estimator.

Ellipticity convention

All three estimators return ellipticities or reduced shears in the *distortion* convention. The quadrupole moments of the surface brightness $I(x, y)$ are

$$Q_{ij} = \frac{\sum_{pq} I(p,q)\,(p_i - \bar{p}_i)(p_j - \bar{p}_j)}{\sum_{pq} I(p,q)}, \tag{11}$$

with flux-weighted centroid $\bar{p}_i$. The two ellipticity components are

$$\epsilon_1 = \frac{Q_{11} - Q_{22}}{D}, \qquad \epsilon_2 = \frac{2\,Q_{12}}{D}, \qquad D = Q_{11} + Q_{22} + 2\sqrt{Q_{11}Q_{22} - Q_{12}^2}. \tag{12}$$

For a pure ellipse with semi-axes $a \geq b$ this gives $|\epsilon| = (a-b)/(a+b)$, with $\epsilon_1 > 0$ ($\epsilon_2 = 0$) for an east–west elongation.

Estimators

`unweighted_ellipticity(image)` Computes (ϵ_1, ϵ_2) from the raw unweighted second moments of the pixel array. The centroid is found from unweighted first moments. Returns `(nan, nan)` when the total flux is non-positive, the determinant $Q_{11}Q_{22} - Q_{12}^2 < 0$ (noise-dominated), or the resulting ellipticity magnitude exceeds unity.

`weighted_ellipticity_raw(image, fwhm_px, max_iter, tol)` Computes (ϵ_1, ϵ_2) from intensity weighted by a circular Gaussian

$$W(x, y) = \exp\left(-\frac{(x - \bar{x}^W)^2 + (y - \bar{y}^W)^2}{2\sigma_w^2}\right), \quad \sigma_w = \frac{\mathrm{FWHM}}{2\sqrt{2\ln 2}}. \tag{13}$$

The weighted centroid $(\bar{x}^W, \bar{y}^W)$ is found iteratively, re-centring the weight on the current estimate until the shift falls below `tol` pixels. The raw ellipticities returned are biased relative to the true reduced shear because the fixed circular weight suppresses the tangential signal; `apply_simulation_calibration` corrects for this.

`hsm_ellipticity(image)` Wraps GalSim's `FindAdaptiveMom` (Hirata and Seljak 2003; Mandelbaum et al. 2005), which iteratively adapts an elliptical Gaussian weight to match the galaxy shape. Returns the reduced shear (g_1, g_2) and the best-fit circular Gaussian size σ_g in pixels.

Calibration

The fixed circular weight introduces a multiplicative shear-response bias: the measured ellipticity $\hat{\epsilon}$ is related to the true reduced shear g by $\hat{\epsilon} \approx R\,g$, where $R < 1$ depends on galaxy size relative to the weight width. `calibrate_weighted_response` estimates $R(\sigma_g)$ by drawing one noiseless GalSim `Exponential` profile at each of twelve half-light radii, applying a known shear $g_1 = 0.05$, and computing $R = \hat{\epsilon}_1/g_1$. The result is a look-up table; `apply_simulation_calibration` divides each galaxy's raw ellipticities by its interpolated $R(\sigma_{g,i})$ using `np.interp`, clamping at the boundary of the calibrated range.

Batch processing

`measure_all_ellipticities(stamps_path)` loads all galaxy stamps from a multi-extension FITS file and applies all three estimators in parallel using a `ThreadPoolExecutor`. The Numba JIT kernels (see Section D) and GalSim's C++ HSM implementation both release the Python GIL, enabling genuine thread-level parallelism.

B.2 `a6cw.shear`

This module implements the full weak-lensing shear pipeline from galaxy positions and ellipticities to a Bayesian posterior on halo mass and source redshift.

Module-level constants

Radial binning is fixed at module import time to ensure consistency across all pipeline functions:

$$\begin{aligned}
&\texttt{N_BINS} = 10,\\
&\texttt{BIN_EDGES} = \mathrm{logspace}(\log_{10} 100,\ \log_{10} 3600,\ 11)\ \text{[arcsec]},\\
&\texttt{BIN_CENTRES}_k = \sqrt{\texttt{BIN_EDGES}_k \cdot \texttt{BIN_EDGES}_{k+1}} \quad \text{(geometric mean)}.
\end{aligned}$$

Pipeline functions

`load_positions(path)` Reads a two-column whitespace-separated text file of (x, y) positions in arcseconds, returning an `(N, 2)` array.

`compute_tangential_ellipticities(halo_pos, source_pos,` ϵ_1`,` ϵ_2`)` Forms all $N_h \times N_s$ halo–source pairs by NumPy broadcasting. The tangential and cross components are computed via trig-free double-angle identities,

$$\cos 2\phi = \frac{dx^2 - dy^2}{r^2}, \qquad \sin 2\phi = \frac{2\,dx\,dy}{r^2}, \tag{14}$$

eliminating `arctan2`, `cos`, and `sin` calls over the full pair array. Returns three flat arrays of length $N_h \times N_s$ containing the separation r [arcsec], the tangential component ϵ_t (E-mode), and the cross component $\epsilon_\times$ (B-mode).

`mean_tangential_shear(r_pairs,` ϵ_t`, bin_edges, weights)` Bins tangential ellipticities into the ten logarithmically-spaced radial bins. In the weak-lensing regime $\langle \epsilon_t \rangle = \gamma_t$, so the mean tangential ellipticity directly estimates the shear. Bin assignment uses a single `np.digitize` pass $[O(N \log B)]$; per-bin sums use `np.bincount`. The unweighted uncertainty is the Bessel-corrected standard error on the mean; an optional inverse-variance weighted path is also provided. Bins with fewer than two valid pairs return `nan`.

`build_nfw_theory(mass, source_redshift, ...)` Evaluates the predicted mean tangential shear at `BIN_CENTRES` for an NFW halo using the `NFWHalo_theory` helper (a thin wrapper around GalSim's `NFWHalo`). Default parameters ($c = 4$, $z_l = 0.3$, $\Omega_m = 0.3$, $\Omega_\Lambda = 0.7$) match the simulation truth.

`log_likelihood(gt_data, gt_err, mass, source_redshift)` Evaluates the Gaussian log-likelihood

$$\ln \mathcal{L} = -\frac{1}{2} \sum_k \left(\frac{\hat{\gamma}_t^k - \gamma_t^{\mathrm{th}}(k)}{\sigma_k} \right)^2, \tag{15}$$

summing only over bins where both data and theory are finite. Returns $-\infty$ if no valid bins remain.

`compute_posterior_grid(gt_data, gt_err, mass_grid, zs_grid)` Evaluates the unnormalised log-posterior on a 2-D (M, z_s) grid under flat priors, distributing cells across OS processes via `ProcessPoolExecutor` (with a serial fallback for grids smaller than 50 cells). Returns an (N_M, N_{z_s}) array.

Plotting utilities

The module also provides `plot_tangential_shear_profiles`, `plot_cross_shear_null_test`, `plot_joint_posterior`, `plot_mass_posterior_fixed_zs`, and `plot_best_fit_overlay`, all writing PNG files to the `outputs/` directory. `plot_joint_posterior` and `plot_mass_posterior_fixed_zs` compute credible-interval boundaries using `scipy.integrate.cumulative_trapezoid`, which performs the CDF integration in a single $O(N)$ pass.

C Test Suite

The package is tested by 82 unit tests spread across two files under `tests/`, runnable with

```
python -m pytest tests/ -v
```

Tests run automatically on every push via a GitHub Actions workflow (`.github/workflows/tests.yml`) using Python 3.11 on `ubuntu-latest`. No files from `data/` or `outputs/` are required; all tests complete in under five seconds.

Design principles

Analytic ground truth where possible. Most tests construct inputs with known mathematical properties and verify the output against a closed-form prediction, rather than comparing to a stored baseline. A failing test therefore points directly to the formula that is wrong.

GalSim profiles for shape estimators. The three estimators operate on `galsim.Image` objects. Tests draw simple noiseless profiles (Gaussians with and without applied shear) using `drawImage` with `method='no_pixel'`, which gives a pixel array matching the analytical profile to machine precision.

Edge cases alongside the happy path. Every estimator test includes a zero-flux (or empty) input that should return `nan`, alongside the positive-signal cases.

Separation of concerns. `test_ellipticity.py` covers only `a6cw.ellipticity`; `test_shear.py` covers only `a6cw.shear`, making failures easy to localise.

C.1 `tests/test_ellipticity.py` (39 tests)

`TestPixelGrids` **(6 tests)** Verifies that `_pixel_grids` returns arrays of the correct shape, that grids sum to zero (symmetric about the centre), that the centre pixel is exactly zero for an odd stamp, and that x varies along columns while y varies along rows.

`TestGaussianWeight` **(4 tests)** Confirms the circular Gaussian weight $W = e^{-(dx^2+dy^2)/2\sigma^2}$ peaks at unity at the origin, falls to $e^{-1/2}$ at one sigma, is symmetric in x and y, and is strictly positive everywhere.

`TestUnweightedEllipticity` **(8 tests)** Checks that a circular Gaussian galaxy gives $(\epsilon_1, \epsilon_2) \approx (0, 0)$ to $< 10^{-6}$; that a zero-flux stamp returns `(nan, nan)`; that axis-aligned sheared Gaussians recover the closed-form value $\epsilon_1 = (\sigma_x - \sigma_y)/(\sigma_x + \sigma_y)$ to 0.1%; that a 45°-rotated galaxy produces pure ϵ_2; that a GalSim Gaussian with $g_1 = 0.3$ returns $\epsilon_1 = 0.3$ to $< 10^{-5}$ relative error (the exact identity $\epsilon_1 = g_1$ for a Gaussian); and that results are bounded by unity.

`TestWeightedEllipticity` **(6 tests)** Analogous tests for the Gaussian-weighted estimator, including a check that a wider weight FWHM increases the raw ϵ_1 response (because it captures more of the galaxy's ellipticity signal).

`TestHSMEllipticity` **(6 tests)** Verifies that circular galaxies give zero HSM shear, that $\sigma_g > 0$, that g_1 and g_2 are recovered to $< 10^{-4}$ relative error on noiseless Gaussian profiles, that empty images return `nan`, and that σ_g increases monotonically with galaxy size.

`TestOLSThroughOrigin` **(4 tests)** Confirms the force-through-origin OLS regression recovers a known slope exactly, recovers a noisy slope to $< 1\%$ with 2000 points, filters `nan` values, and recovers negative slopes.

`TestApplySimulationCalibration` **(5 tests)** Verifies that unit response is the identity; half response doubles ellipticity; `nan` ellipticities remain `nan`; galaxies with `nan` σ_g use the sample median before interpolation; and σ_g values outside the calibration range are clamped to the nearest boundary node.

C.2 `tests/test_shear.py` (43 tests)

`TestModuleConstants` (7 tests) Checks that `BIN_EDGES` has $N_{\rm bins} + 1$ elements, `BIN_CENTRES` are the geometric means of adjacent edges, edges are strictly increasing, the range spans 100–3600 arcsec, and that `COLOURS` and `LABELS` both contain the three method keys.

`TestLoadPositions` (3 tests) Verifies round-trip I/O: write with `np.savetxt`, read with `load_positions`, confirm shape $(N, 2)$ and value preservation (including negative and decimal coordinates).

`TestComputeTangentialEllipticities` (7 tests) Uses three exact analytic cases for the rotation formula:

Source direction	ϕ	ϵ_t	$\epsilon_\times$
East ($x > 0$)	0	$-\epsilon_1$	0
North ($y > 0$)	$\pi/2$	$+\epsilon_1$	0
Northeast ($\phi = \pi/4$, $\epsilon_2 = 1$)	$\pi/4$	$-\epsilon_2$	0

Also checks the 3–4–5 right-triangle separation formula, output shape for $N_h = 7$, $N_s = 13$, and that zero input ellipticity gives zero output.

`TestMeanTangentialShear` (7 tests) Uses a two-bin layout with four known pairs to test: correct mean, pair count, standard error ($\sigma = \sqrt{0.02}/\sqrt{2} = 0.1$ for $\{0.4, 0.6\}$); `nan` for single-pair bins; `nan` exclusion from counts; empty-bin `nan`; and zero error for uniform values.

`TestEstimateShapeNoiseVariance` (4 tests) Confirms known variance ($\{1, -1\}$ gives 2 with `ddof=1`), `nan` exclusion, zero variance for constant arrays, and a `float` return type.

`TestOptimalWeights` (4 tests) Checks equal weights for uniform noise, zero weight for `nan` pairs, positive weight for finite pairs, and $w = 1/\hat{\sigma}_\epsilon^2$.

`TestBuildNFWTheory` (6 tests) Verifies output length equals `N_BINS`; all values positive; profile decreasing with radius; higher mass gives stronger shear at all radii; higher source redshift gives stronger shear; and non-default cosmological parameters are accepted.

`TestLogLikelihood` (5 tests) Confirms a perfect fit gives $\ln \mathcal{L} = 0$; an imperfect fit gives $\ln \mathcal{L} < 0$; a larger residual gives a more negative log-likelihood; `nan` bins are silently excluded; and all-`nan` data returns $-\infty$.

D Performance and Profiling

All benchmarks were measured on WSL2 (Windows Subsystem for Linux), Python 3.11.11, NumPy 2.x, GalSim 2.8.4, and Numba 0.65.1. Wall-clock times are medians over 50–100 repeated calls with one warm-up call excluded. Peak heap memory is measured with Python's `tracemalloc`. Profiling code and plot generation are in `profiling/run_profiling.py`; plots are written to `outputs/profiling/`.

D.1 Optimisations implemented

Eleven optimisations were applied across the two modules and the vendored `nfw_theory.py` file.

1. **Numba JIT for unweighted moments.** The three-pass pixel loop in `unweighted_ellipticity` is compiled by Numba (`@njit(cache=True, nogil=True)`). Intermediate quantities are accumulated in scalar registers, eliminating all temporary NumPy array allocations.

2. **Numba JIT for weighted moments.** The same treatment for `weighted_ellipticity_raw`, with additional micro-optimisations: `inv_2s2` $= 1/(2\sigma^2)$ is precomputed once outside the centroid loop, and convergence is tested as $|\Delta\bar{x}|^2 < \epsilon^2$ to avoid a square root per iteration.

3. **`lru_cache` on pixel grids.** The coordinate offset grids are cached by stamp shape via `functools.lru_cache(maxsize=32)`, so repeated calls on same-size stamps incur no allocation cost.

4. **`ThreadPoolExecutor` in `measure_all_ellipticities`.** Both Numba JIT kernels and GalSim's C++ HSM implementation release the Python GIL, so a thread pool achieves genuine parallelism across galaxy stamps.

5. **Single GalSim draw per HLR in `calibrate_weighted_response`.** Noiseless stamps with fixed parameters are fully deterministic; the original loop drew 1000 pixel-identical stamps and averaged them. Reducing to one draw per HLR gives a $\sim$1000$\times$ reduction in GalSim renders (12 000 $\to$ 12).

6. **Trig-free double-angle identities in `compute_tangential_ellipticities`.** $\cos 2\phi = (dx^2 - dy^2)/r^2$ and $\sin 2\phi = 2\,dx\,dy/r^2$ eliminate `arctan2`, `cos`, and `sin` calls over the $N_h \times N_s$ pair array.

7. **`np.digitize` + `np.bincount` in `mean_tangential_shear` (unweighted).** A single $O(N \log B)$ digitize pass replaces B repeated boolean masks [$O(NB)$ total].

8. **`np.bincount` weighted path.** Per-bin loops for the inverse-variance weighted sums are replaced with vectorised `np.bincount` calls.

9. **`ProcessPoolExecutor` in `compute_posterior_grid`.** The NFW theory evaluation holds the GIL; separate OS processes bypass it.

10. **`cumulative_trapezoid` for CDF computation.** The $O(N^2)$ list comprehension `[np.trapezoid(p[:k+1` `x[:k+1]) for k in range(N)]` is replaced by `scipy.integrate.cumulative_trapezoid` [$O(N)$], which also avoids accumulation of floating-point rounding from repeated prefix sums.

11. **Vectorised `NFWHalo.__gamma` in `nfw_theory.get_tangential_shear`.** GalSim's private `__gamma(r, ks)` accepts arrays; broadcasting `ks_arr = np.full_like(rs, ks)` replaces a Python loop over 10 radial bins with one vectorised C-level call.

D.2 Results

Table 1 summarises the before/after wall-clock times at representative input sizes, and Table 2 summarises peak heap memory for the two moment estimators.

The 18 KB residual after optimisation equals the input array itself ($48\times48\times8$ bytes = 18 KB), confirming that the Numba JIT kernels allocate no intermediate arrays.

The dominant gain is `calibrate_weighted_response`: the $\sim$230$\times$ speedup (not 1000$\times$) reflects fixed per-HLR overhead in GalSim cosmology initialisation and pixel convolution that is unaffected by the draw-count reduction. The posterior-grid speedup of 9.3$\times$ decomposes into approximately 6$\times$ from the vectorised NFW call (opt. 11) and 1.6$\times$ additional from `ProcessPoolExecutor` (opt. 9) at 2 500 cells. The coursework grid ($100 \times 50 = 5\,000$ cells) completes in $\approx$0.35 s versus the original $\approx$2.5 s (7$\times$).

Table 1: Wall-clock performance before and after all optimisations (medians). The `calibrate_weighted_response` entry combines optimisations 5 and 4; the posterior grid entry combines optimisations 9 and 11.

Routine (input size)	Before	After	Speedup
`unweighted_ellipticity` (48 × 48 px)	0.030 ms	0.007 ms	4×
`weighted_ellipticity_raw` (48 × 48 px)	0.066 ms	0.025 ms	2.6×
`hsm_ellipticity` (48 × 48 px)	0.184 ms	0.175 ms	1.1×
`calibrate_weighted_response` (12 HLRs)	∼14 s	60 ms	∼230×
`compute_tangential_ellipticities` (10^5 sources)	84 ms	39 ms	2.2×
`build_nfw_theory` (10 bins)	0.421 ms	0.070 ms	6×
`compute_posterior_grid` (2 500 cells)	1 217 ms	131 ms	9.3×

Table 2: Peak heap memory for the two pixel-based moment estimators at 48 × 48 pixels. HSM (C++) is unchanged.

Estimator	Before	After	Reduction
Unweighted moments	127 KB	18 KB	7×
Gaussian-weighted moments	181 KB	18 KB	10×
HSM (GalSim)	13 KB	13 KB	unchanged

D.3 Scaling behaviour

The ellipticity estimators scale as $O(N_{\rm px})$ in the number of pixels ($N_{\rm px} = N^2$ for an $N \times N$ stamp). At small stamps the measured log–log slope is below unity (0.84 for unweighted, 1.12 for weighted) because fixed Numba dispatch overhead dominates at a few hundred pixels; above 128 × 128 all estimators approach the asymptotic linear regime. The sub-linear slope for unweighted and super-linear slope for weighted at small stamps reflect the difference in per-iteration work: the unweighted kernel performs fewer floating-point operations per pixel, so the relative overhead fraction is larger, while the weighted kernel's extra centroid iteration loop makes per-pixel cost slightly larger than linear at small N.

`compute_tangential_ellipticities` scales as $O(N_h \cdot N_s)$, with a measured log–log slope of ≈1.0 confirming linear scaling with source count at fixed halo count. At 10^5 sources and 10 halos, peak memory is ≈77 MB (10^6 double-precision pairs × three arrays).

The posterior grid scales as $O(N_M \cdot N_{z_s}/N_{\rm cores})$ with `ProcessPoolExecutor`. At 2 500 cells the per-cell amortised cost is 0.05 ms, compared to 0.49 ms in the original serial implementation.